Systematic Conversion of Linear Recurrences to Matrix Exponentiation

This document outlines the systematic **Zero-Indexed Shift** framework for converting recurrence relations into matrix exponentiation. This method defines the state equation as $V_n = T^n V_0$, where, for pure linear recurrences, the final answer always lands in a fixed row, which avoids off-by-one errors and negative indices.

1 Pure Linear Recurrences

These are homogeneous equations where the next state depends exclusively on linear combinations of previous states.

1.1 The Universal Blueprint

For a linear recurrence of order k:

$$f(n) = c_1 f(n-1) + c_2 f(n-2) + \cdots + c_k f(n-k) \qquad (n \geq k)$$

1. **The Base Vector (V_0):** A $k \times 1$ column vector containing the first k known base cases, ordered from newest $(k-1)$ at the top to oldest (0) at the bottom.

$$V_0 = \begin{bmatrix} f(k-1) \\ f(k-2) \\ \vdots \\ f(0) \end{bmatrix}$$

2. **The Transition Matrix (T):** A $k \times k$ matrix. The first row holds the coefficients $c_1, \ldots, c_k$. The remaining rows consist of a $(k-1) \times (k-1)$ identity matrix in the left $k-1$ columns, followed by a zero column; this shifts every state down by one position.

$$T = \begin{bmatrix} c_1 & c_2 & c_3 & \ldots & c_k \\ 1 & 0 & 0 & \ldots & 0 \\ 0 & 1 & 0 & \ldots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & \ldots & 0 & 1 & 0 \end{bmatrix}$$

3. **The Result (V_n):** Compute $T^n V_0$ by fast exponentiation in $O(k^3 \log n)$ time. The target answer $\mathbf{f(n)}$ will consistently drop to the **bottom row** (index $k-1$).

$$V_n = T^n V_0 = \begin{bmatrix} f(n+k-1) \\ f(n+k-2) \\ \vdots \\ \mathbf{f(n)} \end{bmatrix}$$

1.2 Example 1: Classic Fibonacci

Equation: $f(n) = f(n-1) + f(n-2)$
Order (k): 2
Coefficients: $c_1 = 1, c_2 = 1$

$$\begin{bmatrix} f(n+1) \\ \mathbf{f(n)} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} f(1) \\ f(0) \end{bmatrix}$$

1.3 Example 2: Third-Order Recurrence with Arbitrary Coefficients

Equation: $f(n) = 2f(n-1) + 3f(n-2) + 4f(n-3)$
Order (k): 3
Coefficients: $c_1 = 2, c_2 = 3, c_3 = 4$

$$\begin{bmatrix} f(n+2) \\ f(n+1) \\ \mathbf{f(n)} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}^n \begin{bmatrix} f(2) \\ f(1) \\ f(0) \end{bmatrix}$$

1.4 Example 3: Summing with 1 and m

Equation: $f(n) = f(n-1) + f(n-m)$, with $m \geq 2$
Order (k): m
Coefficients: $c_1 = 1,\ c_2 = \cdots = c_{m-1} = 0,\ c_m = 1$
(For $m = 1$ the recurrence degenerates to $f(n) = 2f(n-1)$.)

$$\begin{bmatrix} f(n+m-1) \\ \vdots \\ f(n+1) \\ \mathbf{f(n)} \end{bmatrix} = \begin{bmatrix} 1 & 0 & \dots & 0 & 1 \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & \dots & 0 & 1 & 0 \end{bmatrix}^n \begin{bmatrix} f(m-1) \\ \vdots \\ f(1) \\ f(0) \end{bmatrix}$$

2 Non-Homogeneous Recurrences

When appending constants, polynomials, or geometric terms, the state vector must be expanded to track and continuously update those terms. Because the vector structure deviates from the pure blueprint, the row in which $f(n)$ appears depends on how the state is constructed and must be identified case by case. Each example below states its recurrence for $n \geq 1$ (or $n \geq 2$) together with the initial values.

2.1 The Universal Blueprint

Every row of the transition matrix T answers one question: *"How does this component of my state evolve from step i to step $i+1$?"*

1. **Choose the State Vector Components.** The state vector must include every quantity needed to fully describe the next step. Use the following rules to decide what to include:

Term in recurrence	**Components to add to the state**
Order-k recurrence	Last k values: $f(i+k-1), \ldots, f(i)$
Constant C	A single dummy 1 (maps to itself)
Linear n	i and 1 (since $i+1 = 1 \cdot i + 1 \cdot 1$)
Quadratic n^2	i^2, i, and 1 (since $(i+1)^2 = i^2 + 2i + 1$)
Geometric c^n	c^i alone (since $c^{i+1} = c \cdot c^i$)

   If any component $s_j(i+1)$ cannot be written as a linear combination of the current state, add more components until it can.

2. **Derive Each Row of T.** Set up the general equation $S(i+1) = T \cdot S(i)$:

$$\begin{bmatrix} s_1(i+1) \\ s_2(i+1) \\ \vdots \\ s_m(i+1) \end{bmatrix} = T \times \begin{bmatrix} s_1(i) \\ s_2(i) \\ \vdots \\ s_m(i) \end{bmatrix}$$

   Then for each component $s_j(i+1)$, expand it as a linear combination of the *current* state components. The coefficients directly give row j of T:

$$s_j(i+1) = \underbrace{a_{j1}}_{\text{col } 1} \cdot s_1(i) + \underbrace{a_{j2}}_{\text{col } 2} \cdot s_2(i) + \cdots + \underbrace{a_{jm}}_{\text{col } m} \cdot s_m(i) \quad \Rightarrow \quad \text{row } j \text{ of } T = [\, a_{j1},\ a_{j2},\ \ldots,\ a_{jm} \,]$$

   This works because a matrix–vector product $T \cdot S(i)$ computes each output entry as exactly a dot product of one row of T with the full vector $S(i)$. Reading the equation "backwards" from the recurrence gives the rows directly.

3. **Set the Base Vector.** Evaluate each state component at the starting index to get V_0.

2.2 Example 4: Adding a Constant

Equation: $f(n) = f(n-1) + f(n-2) + C$ for $n \geq 2$
Strategy: Append a dummy state of 1 at the bottom of the vector to continually feed the constant C into the new state. Following the zero-indexed shift framework, the base vector V_0 holds the first two known values $f(1)$ and $f(0)$, so that $f(n)$ lands at the **second row** of V_n.

$$\begin{bmatrix} f(n+1) \\ \mathbf{f(n)} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & C \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}^n \begin{bmatrix} f(1) \\ f(0) \\ 1 \end{bmatrix}$$

2.3 Example 5: Adding a Variable (n)

Equation: $f(n) = 2f(n-1) + n$ for $n \geq 1$
Strategy: To step $i \to i+1$, the vector must track the current index i and a constant 1. Here $f(n)$ lands in the **first row** of V_n.

$$\begin{aligned} f(i+1) &= 2f(i) + (i+1) = 2f(i) + 1 \cdot i + 1 \cdot 1 \\ (i+1) &= 0 \cdot f(i) + 1 \cdot i + 1 \cdot 1 \\ 1 &= 0 \cdot f(i) + 0 \cdot i + 1 \cdot 1 \end{aligned}$$

$$\begin{bmatrix} \mathbf{f(n)} \\ n \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}^n \begin{bmatrix} f(0) \\ 0 \\ 1 \end{bmatrix}$$

2.4 Example 6: Adding a Geometric Progression (c^n)

Equation: $f(n) = f(n-1) + 3^n$ for $n \geq 1$
Strategy: To transition $3^i \to 3^{i+1}$, multiply the geometric state by its base. Here $f(n)$ lands in the **first row** of V_n.

$$\begin{aligned} f(i+1) = f(i) + 3^{i+1} &= 1 \cdot f(i) + 3 \cdot 3^i \\ 3^{i+1} &= 0 \cdot f(i) + 3 \cdot 3^i \end{aligned}$$

$$\begin{bmatrix} \mathbf{f(n)} \\ 3^n \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 0 & 3 \end{bmatrix}^n \begin{bmatrix} f(0) \\ 3^0 \end{bmatrix}$$

2.5 Example 7: Adding a Polynomial (n^2)

Equation: $f(n) = f(n-1) + n^2$ for $n \geq 1$
Strategy: To transition $i^2 \to (i+1)^2$, use $(i+1)^2 = i^2 + 2i + 1$. The state vector must track i^2, i, and 1. Here $f(n)$ lands in the **first row** of V_n.

$$\begin{aligned} f(i+1) = f(i) + (i+1)^2 &= 1 \cdot f(i) + 1 \cdot i^2 + 2 \cdot i + 1 \cdot 1 \\ (i+1)^2 &= 1 \cdot i^2 + 2 \cdot i + 1 \cdot 1 \\ (i+1) &= 1 \cdot i + 1 \cdot 1 \\ 1 &= 1 \cdot 1 \end{aligned}$$

$$\begin{bmatrix} \mathbf{f(n)} \\ n^2 \\ n \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}^n \begin{bmatrix} f(0) \\ 0^2 \\ 0 \\ 1 \end{bmatrix}$$